OMDS-ModB2-Week9-Zucker-Scharff-Thomas

March 23, 2026

1 Week 9 - Multivariate Analysis, part 2

2 1. Lesson - no lesson this week

3 2. Weekly graph question

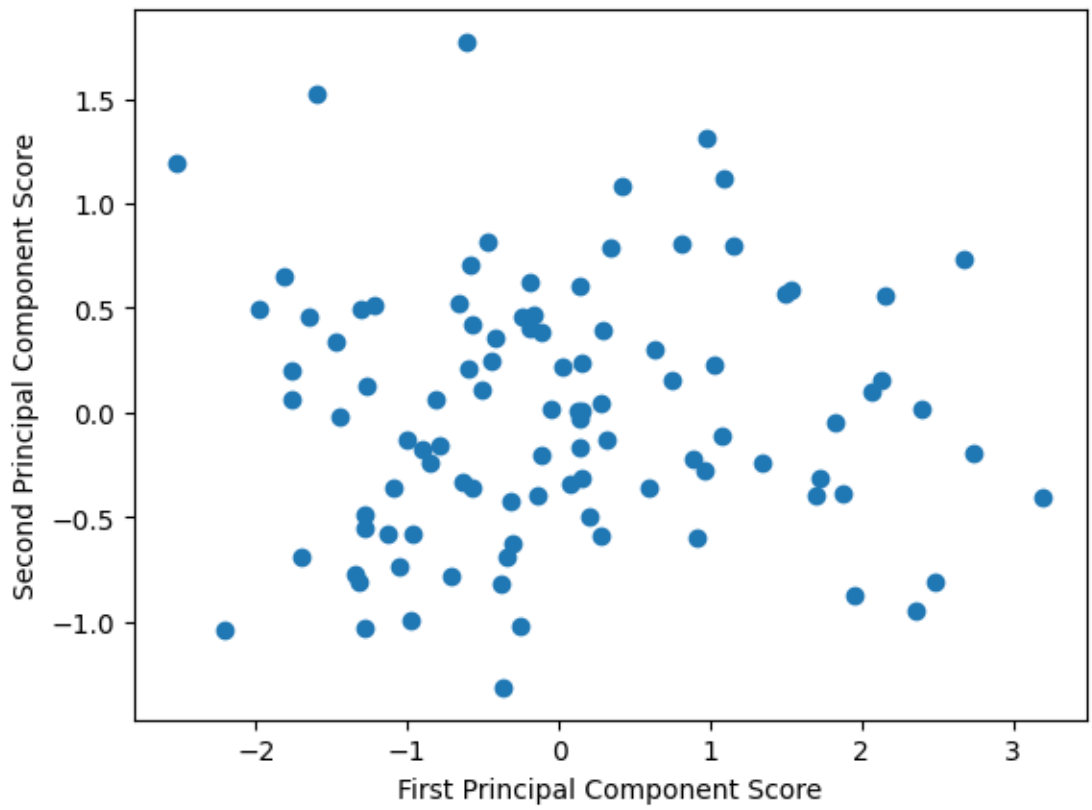
The graph below plots the first two principal component scores in a scatter plot. What can be said about the three outliers in the upper left corner of the graph? Is their first principal component score high or low? What about their second principal component score? What does that mean about their values in series_1, series_2, and series_3? It seems to me that you can say something about series_3 (what can you say?) but you may have a harder time saying something about series_1 and series_2, and an almost impossible time saying anything about the relative values of series_1 and series_2. Why is that? How are series_1 and series_2 related, according to how they were created? If you like, try drawing a pairplot for all three series and see what you get.

Overall, what are the advantages and disadvantages of the graph below? Does it show anything interesting?

[3]:
```
import numpy as np
import pandas as pd
from sklearn import decomposition
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels as sm

np.random.seed(0)
num_points = 100
series_1 = np.random.normal(loc = 2, scale = 0.5, size = num_points)
series_2 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.1, size =␣
↪num_points))
series_3 = series_1 * (1 + np.random.normal(loc = 0, scale = 0.5, size =␣
↪num_points))
df = pd.DataFrame({'ser1': series_1, 'ser2': series_2, 'ser3': series_3})
df = df - df.mean() # set mean to zero, so we don't have to subtract mean from␣
↪the principal component scores

pca3 = decomposition.PCA(n_components = 3)
pca3.fit(df)
```

```
print(pca3.explained_variance_ratio_)
print(pca3.components_)

first_principal_component_score = df.dot(pca3.components_[0])
second_principal_component_score = df.dot(pca3.components_[1])
plt.scatter(first_principal_component_score, second_principal_component_score)
plt.xlabel("First Principal Component Score")
plt.ylabel("Second Principal Component Score");
```

```
[0.79916477 0.18990532 0.01092991]
[[ 0.26541493  0.30096233  0.91595665]
 [ 0.60337553  0.6891417  -0.40127506]
 [ 0.75199261 -0.65917023 -0.00131519]]
```



3.0.1 Answers

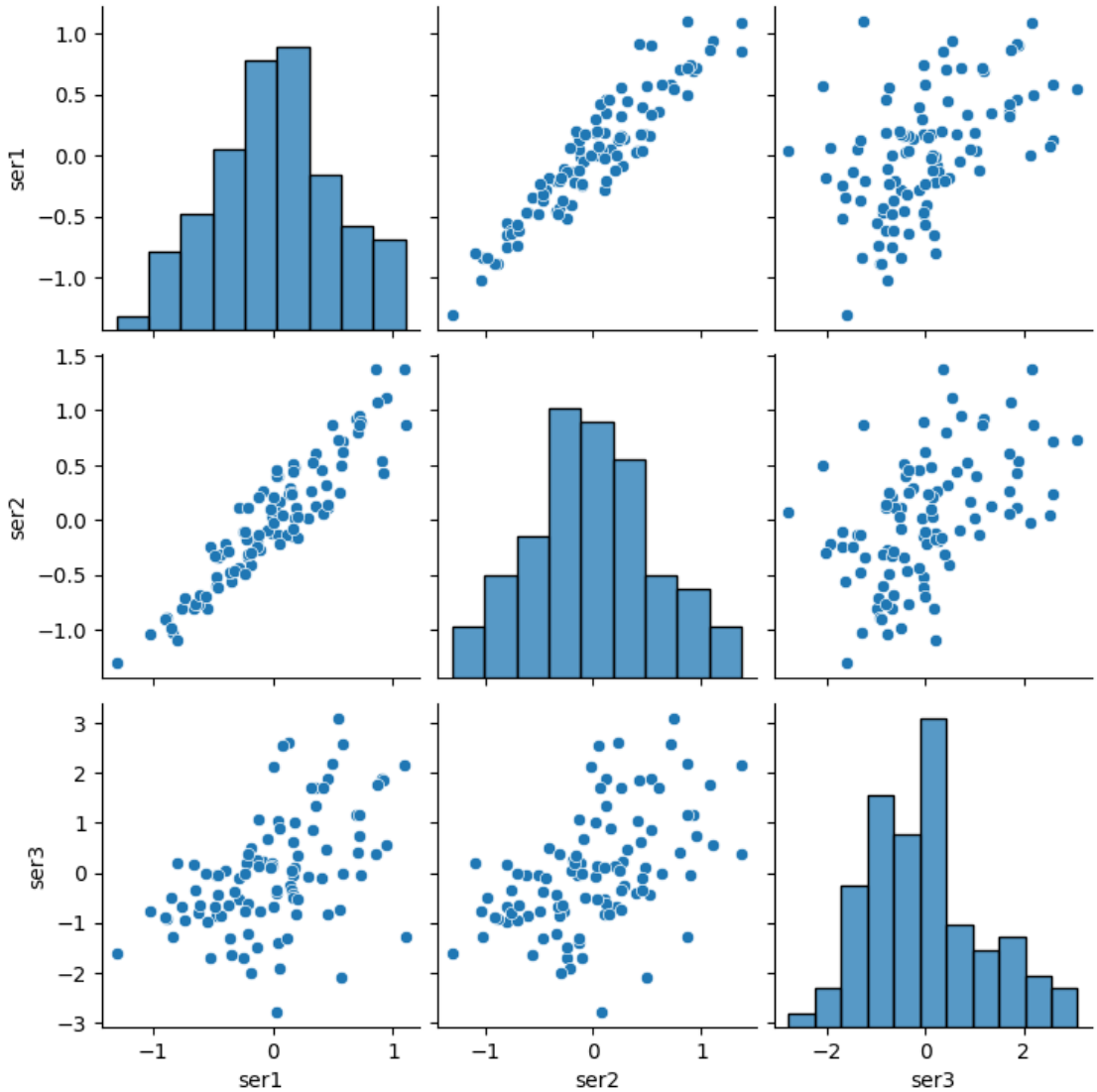
The top outliers are not explained by the first two principal components. A third principal component would most likely explain those outliers. These outliers are likely to have a high score for the first principal component as they will be farther along the PCA axis.

The way the series were created based series_2 and series_3 on series_1 with a scale differential. In the case of serie 2 the scale offset is 1, while in the case of series 3, the scale offset is 0.5.

There is a linear relationship between series 1 and series 2. While series 3 seems to outside that relationship. in the pairplot below it seems that Series_3 is sometimes represented as series_2 inverted.

The graph does attempt to show correlations between the series. It is better seen in the pairplot of the dataframe below. The graph does show very obviously outliers. Because this is a PCA 1 vs PCA 2 graph, you are not seeing the actual data, but rather a graph of how the first two PCA components represent the underlying data.

[4]:
```
sns.pairplot(df);
```



4 3. Working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do: Draw heatmaps.

Draw bubble plots.

Perform Principal Component Analysis to find out the directions in which the data varies. Can you represent the data using only its projection onto its first principal component, using the methods described in Week 8? How much of the variance would this capture?

Try performing linear regression analysis using different sets of features. Which features seem most likely to be useful to predict other features?

Conclusions: Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If the data are not useable, find some new data!

Do you see any outliers? (Data points that are far from the rest of the data).

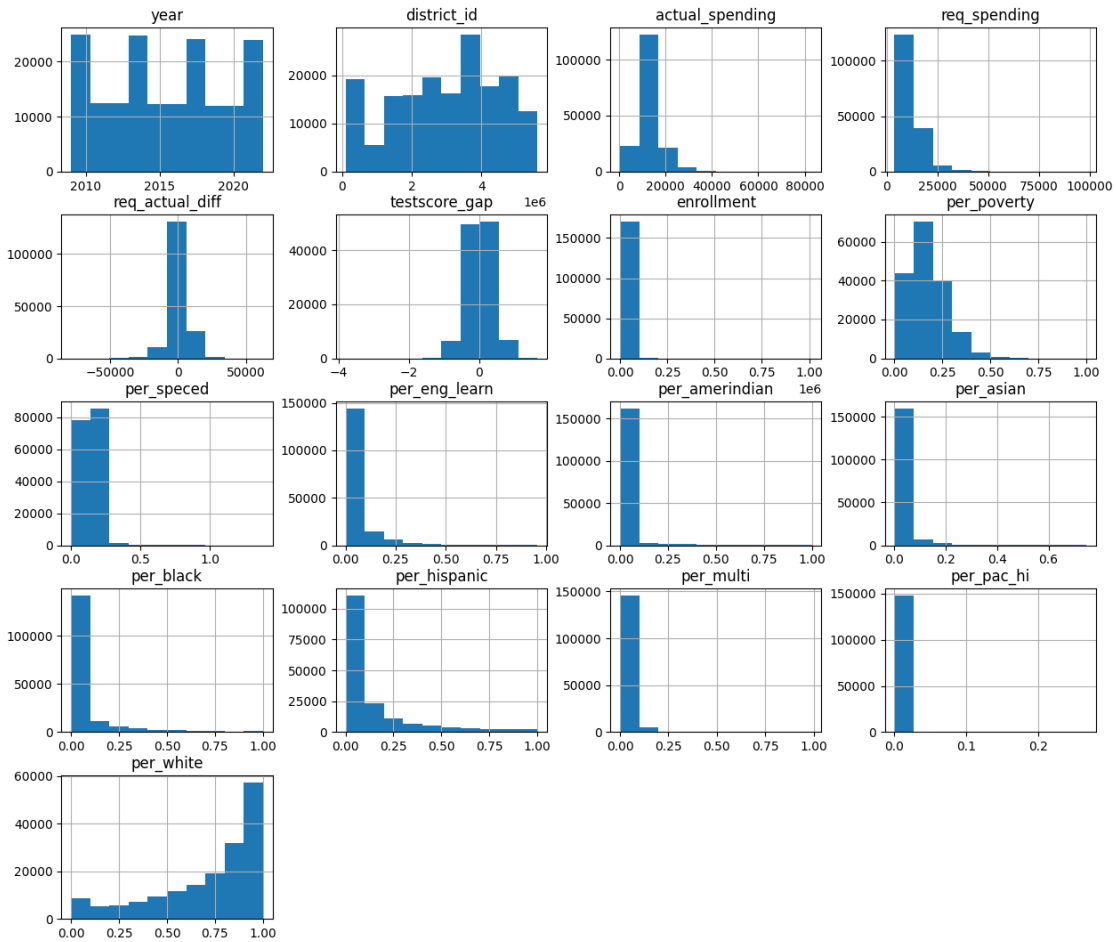
Does the Principal Component Analysis suggest a way to represent the data using fewer dimensions than usual - using its first one or two principal component scores, perhaps?

Try using your correlation information from previous weeks to help choose features for linear regression.

[5]:
```
dcd25_renamed = pd.read_csv('dcd25_renamed.csv')
```

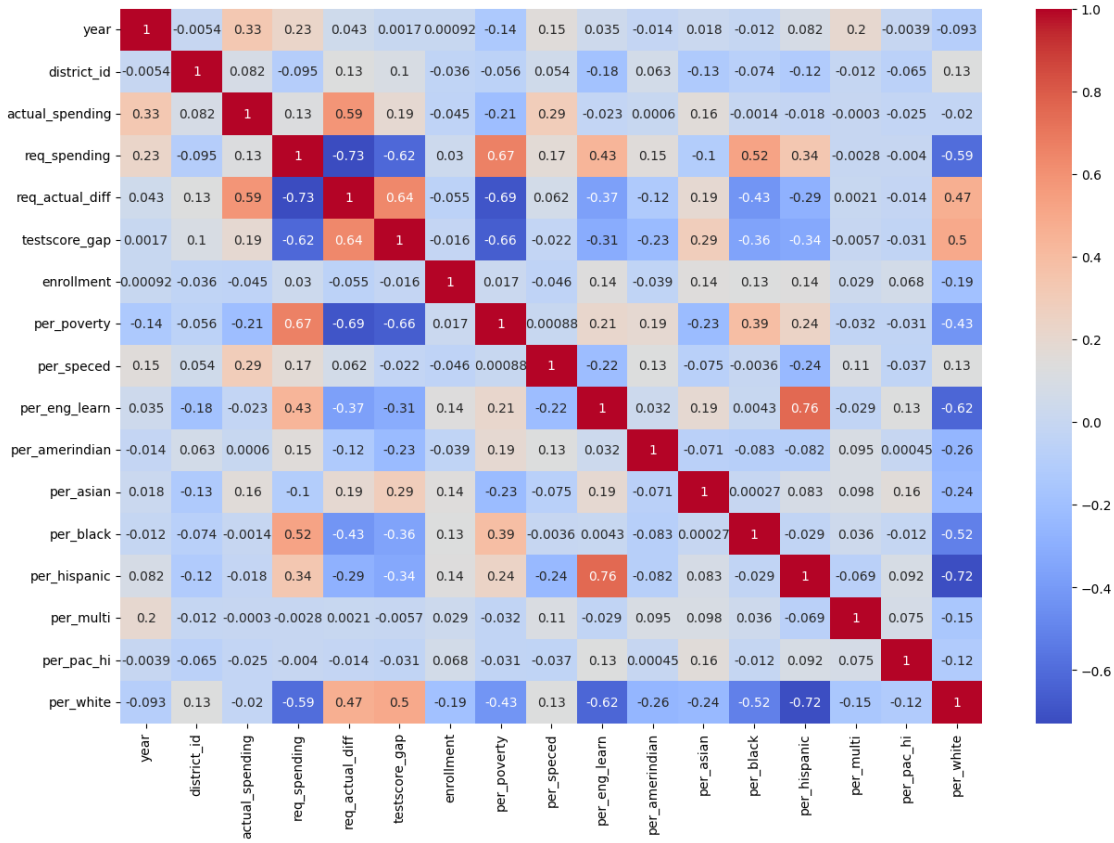
Histogram of dataset

[6]:
```
figsize = (15, 13)
dcd25_renamed.hist(figsize=figsize);
```



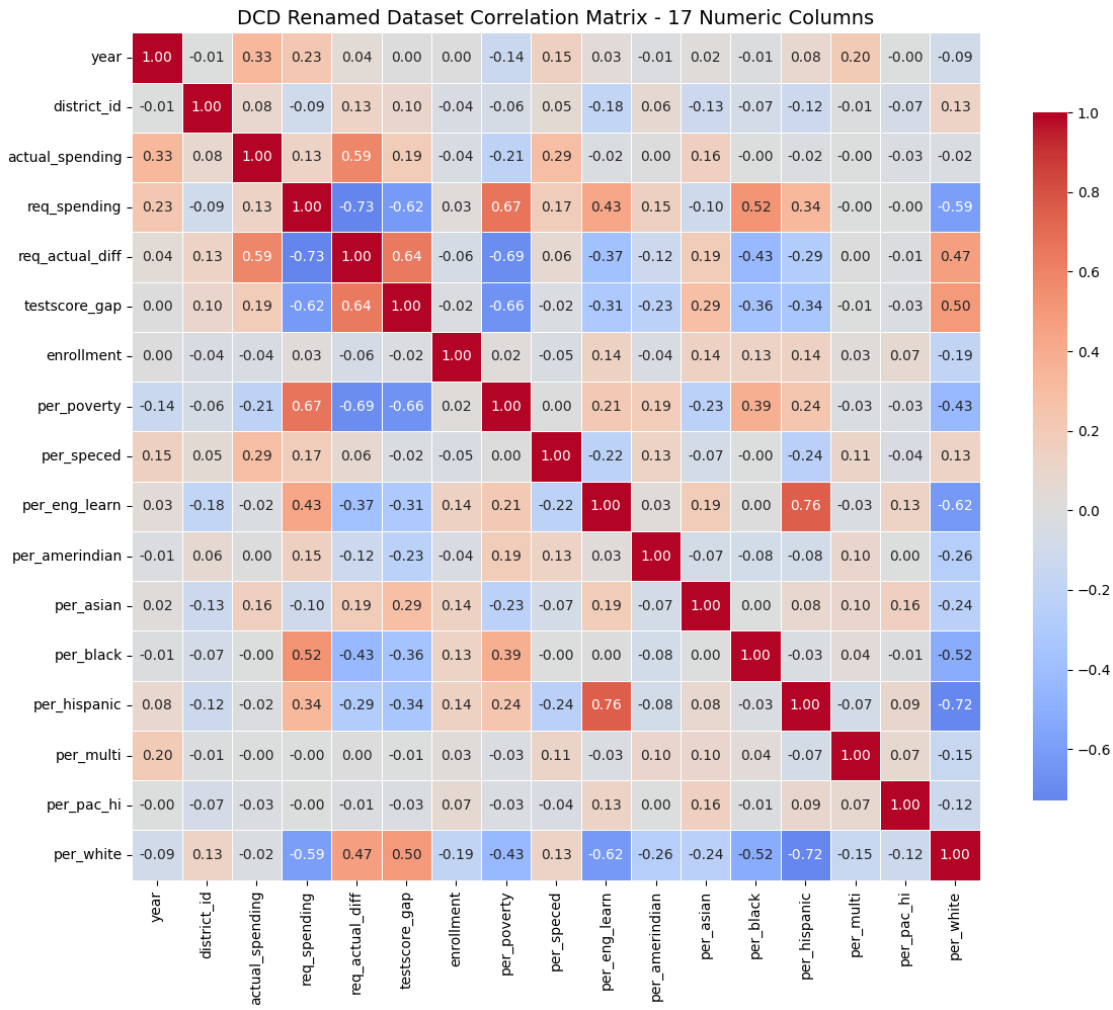
Correlation Heatmap

[7]:
```
# correlation heatmap of dcd25 after drops and renames
dcd25 = pd.read_csv('dcd25_renamed.csv')
dcd25_numeric = dcd25.select_dtypes(include = [np.number])
plt.figure(figsize = (15, 10))
sns.heatmap(dcd25_numeric.corr(), annot = True, cmap = 'coolwarm');
```



[8]:
```
column_to_start = 0
column_to_end = 17

dcd25_renamed_numeric_columns = dcd25_renamed.select_dtypes(include=['int64',␣
↪'float64']).columns.tolist()
dcd25_renamed_numeric_data = dcd25_renamed[dcd25_renamed.columns.
↪intersection(dcd25_renamed_numeric_columns)].iloc[:, column_to_start:
↪column_to_end]

# Calculate correlation matrix for numeric columns
corr_matrix = dcd25_renamed_numeric_data.corr(numeric_only=True)

# Create correlagram
plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', center=0,
            fmt='.2f', square=True, linewidths=0.5, cbar_kws={"shrink": 0.8})
plt.title('DCD Renamed Dataset Correlation Matrix - {0} Numeric Columns'.
↪format(column_to_end - column_to_start), fontsize=14)
plt.tight_layout()
plt.show()
```

```
plt.close()
```



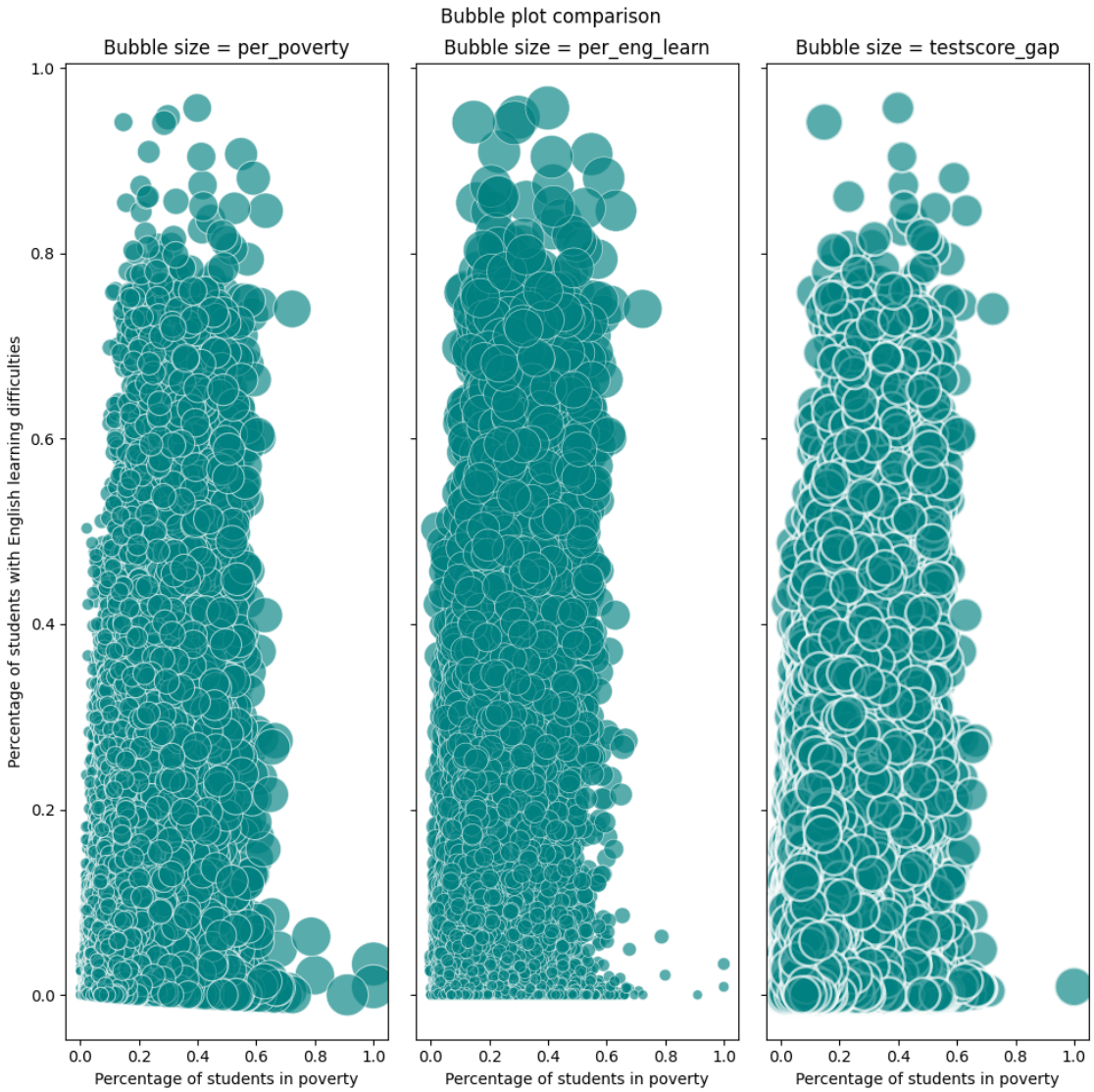
This graph represents all the numeric columns in the dcd25_renamed dataset, which are from column index 0 to 16 (inclusive). The heatmap shows the pairwise correlation coefficients between these numeric columns, with values ranging from -1 (perfect negative correlation) to +1 (perfect positive correlation). The color intensity indicates the strength of the correlation, with darker colors representing stronger correlations.

This shows a strong correlation (0.76) between the percent of hispanic students within a district (per_hispanic) and the percent of English language learners (per_eng_learn). This is not surprising, and should be investigated further.

In a smaller dataset, like the district cost dataset - which has only 17 numeric features, I believe that an annotated correlation matrix gives one more information than a correlogram.

Bubble plots

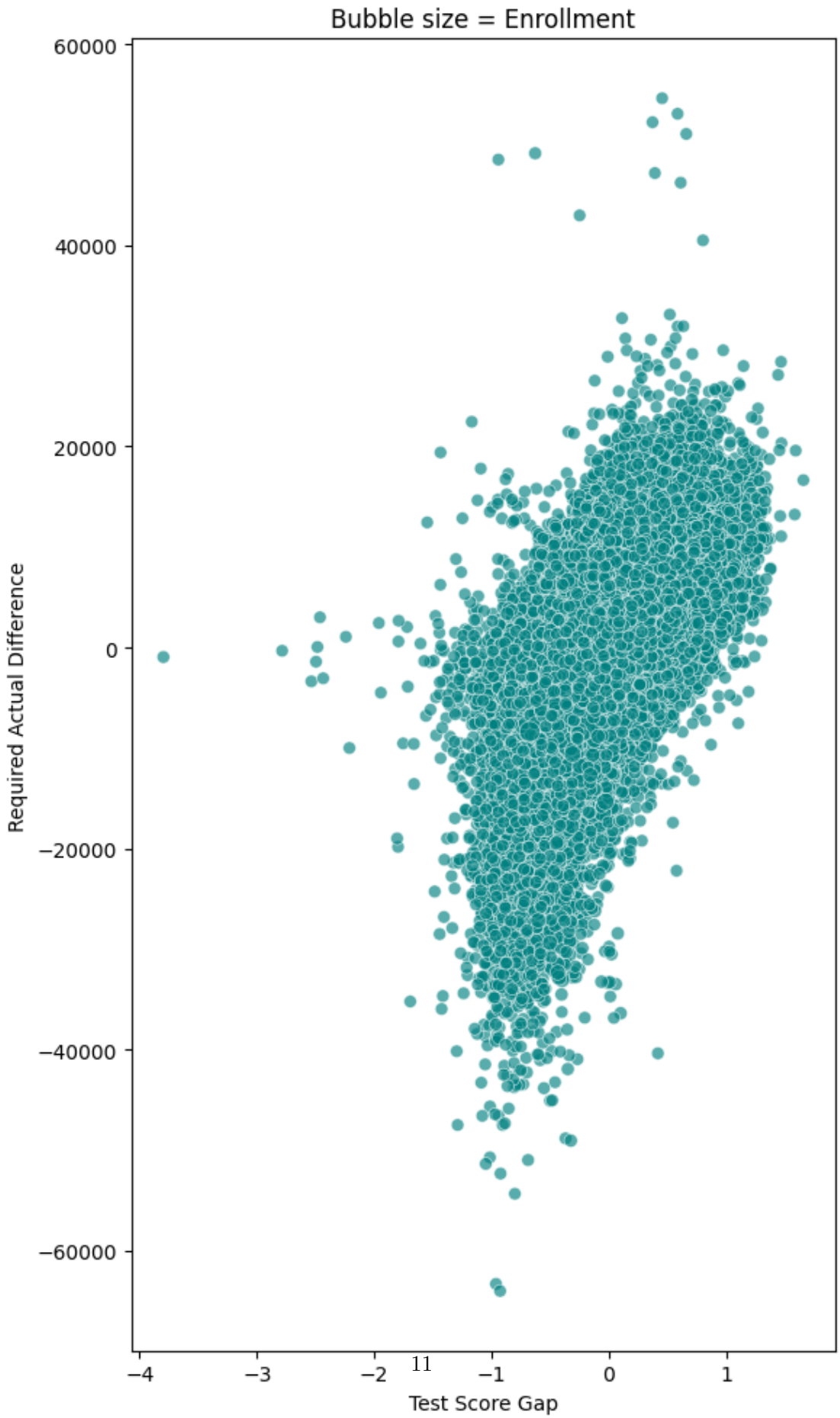
[9]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Bubble plots with x=milk, y=cheese, and bubble size based on each candidate␣
↪variable
size_columns = ["per_poverty", "per_eng_learn", "testscore_gap"]
fig, axes = plt.subplots(1, 3, figsize = (10, 10), sharex = True, sharey = True)

for ax, size_col in zip(axes, size_columns):
    sns.scatterplot(
        data = dcd25_renamed,
        x = "per_poverty",
        y = "per_eng_learn",
        size = size_col,
        sizes = (40, 800),
        alpha = 0.65,
        color = "teal",
        legend = False,
        ax = ax
    )
    ax.set_title(f"Bubble size = {size_col}")
    ax.set_xlabel("Percentage of students in poverty")
    ax.set_ylabel("Percentage of students with English learning difficulties")

fig.suptitle("Bubble plot comparison")
plt.tight_layout()
plt.show()
```



testscore gap vs req_actual spending diff with enrollment as size

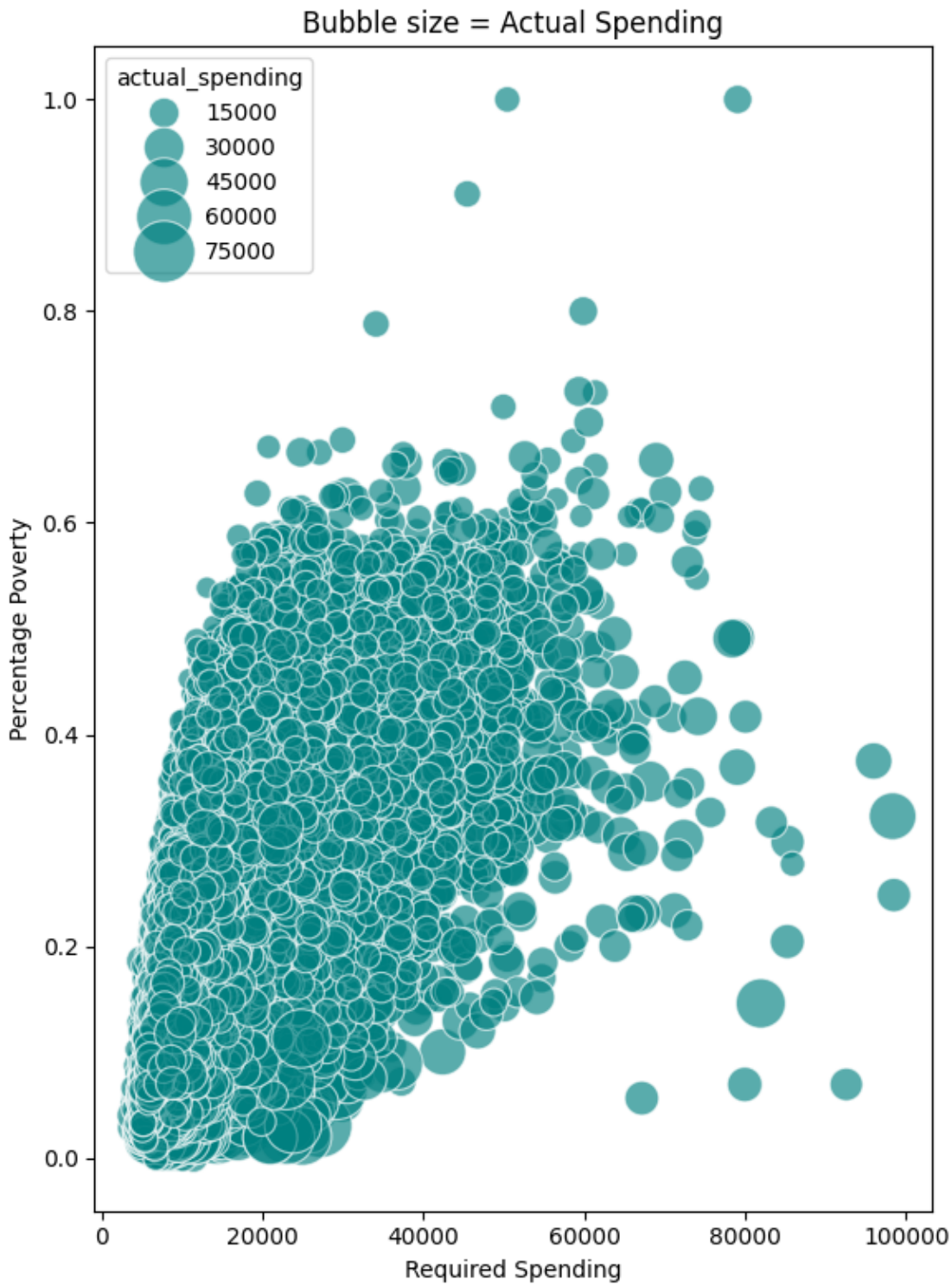
[10]:
```
fig, ax = plt.subplots(figsize=(6, 10))

sns.scatterplot(
    data=dcd25,
    x="testscore_gap",
    y="req_actual_diff",
    size="enrollment",
    sizes=(40, 800),
    alpha=0.65,
    color="teal",
    legend=False,
```

```
    ax=ax
)

ax.set_title("Bubble size = Enrollment")
ax.set_xlabel("Test Score Gap")
ax.set_ylabel("Required Actual Difference")

plt.tight_layout()
plt.show()
```


Bubble size = Enrollment
60000
40000
20000
0
−20000
−40000
−60000
Required Actual Difference
−4
−3
−2
−1
0
1
Test Score Gap

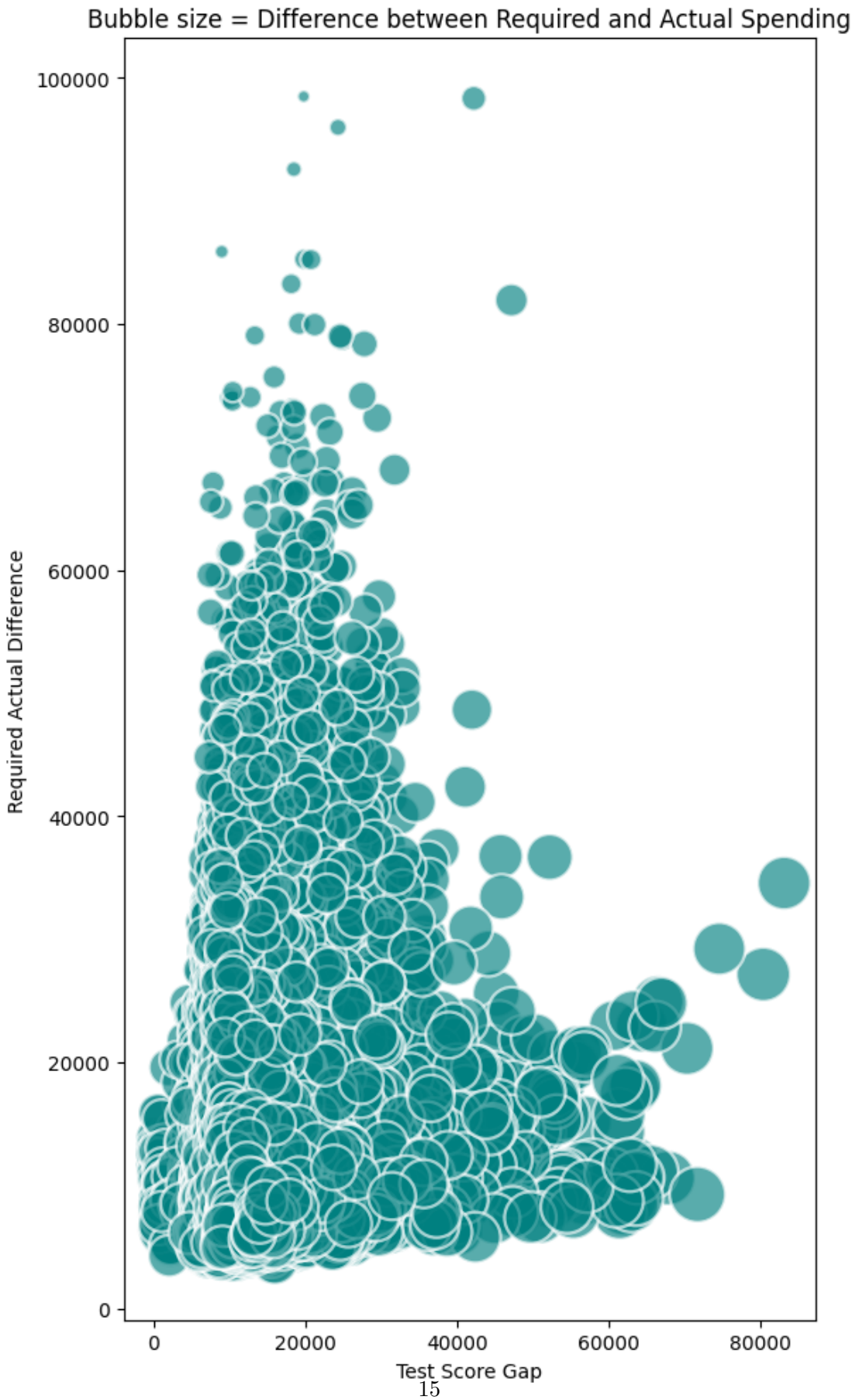
Required Spending vs percent poverty with actual spending as bubble size

[11]:
```
fig, ax = plt.subplots(figsize=(6, 8))

sns.scatterplot(
    data=dcd25,
    x="req_spending",
    y="per_poverty",
    size="actual_spending",
    sizes=(40, 800),
    alpha=0.65,
    color="teal",
    legend=True,
    ax=ax
)

ax.set_title("Bubble size = Actual Spending")
ax.set_xlabel("Required Spending")
ax.set_ylabel("Percentage Poverty")

plt.tight_layout()
plt.show()
```

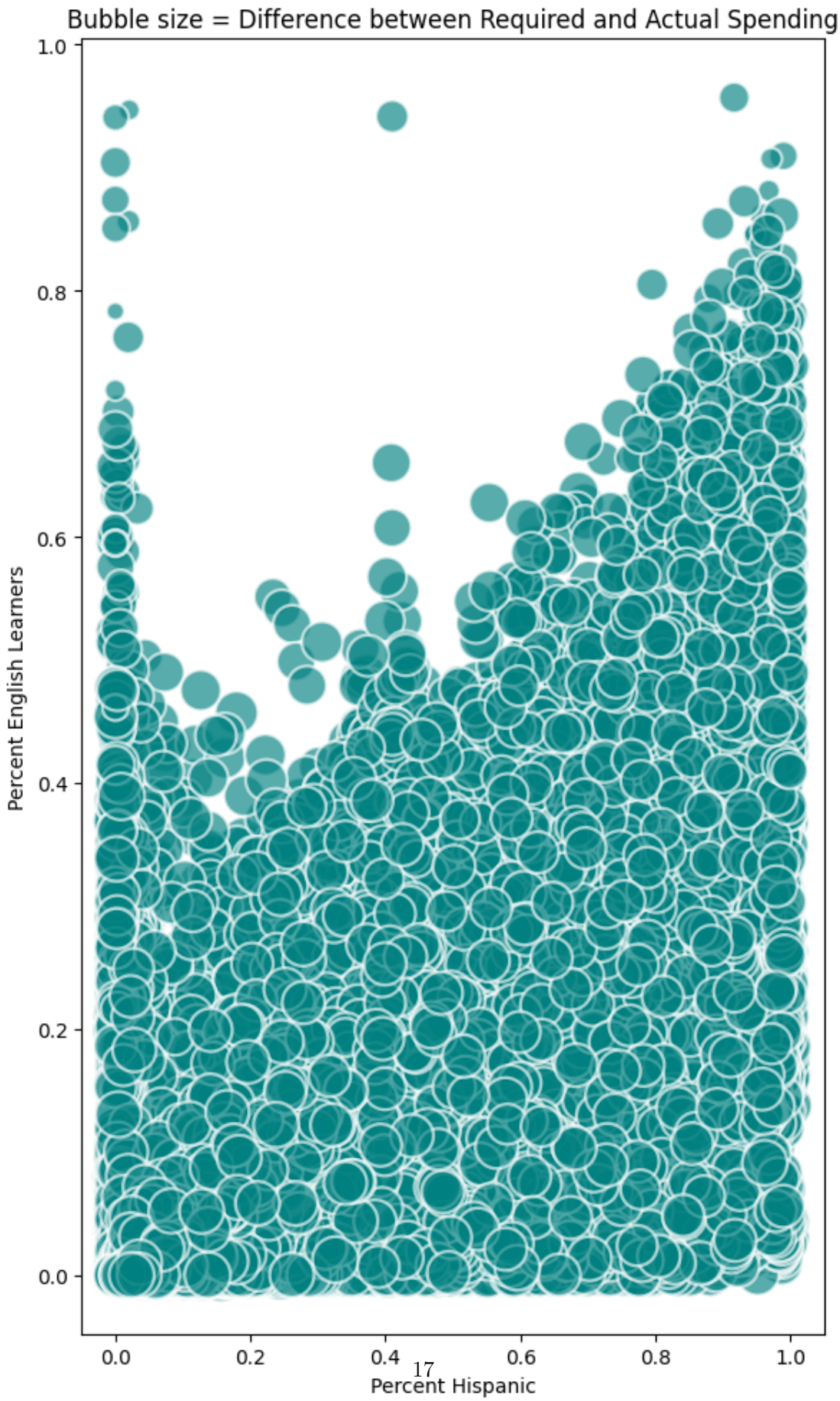


actual vs req spending with req_act_diff as size

[12]:
```
fig, ax = plt.subplots(figsize=(6, 10))

sns.scatterplot(
    data=dcd25,
    x="actual_spending",
    y="req_spending",
    size="req_actual_diff",
    sizes=(40, 800),
    alpha=0.65,
    color="teal",
    legend=False,
    ax=ax
)

ax.set_title("Bubble size = Difference between Required and Actual Spending")
ax.set_xlabel("Test Score Gap")
ax.set_ylabel("Required Actual Difference")

plt.tight_layout()
plt.show()
```

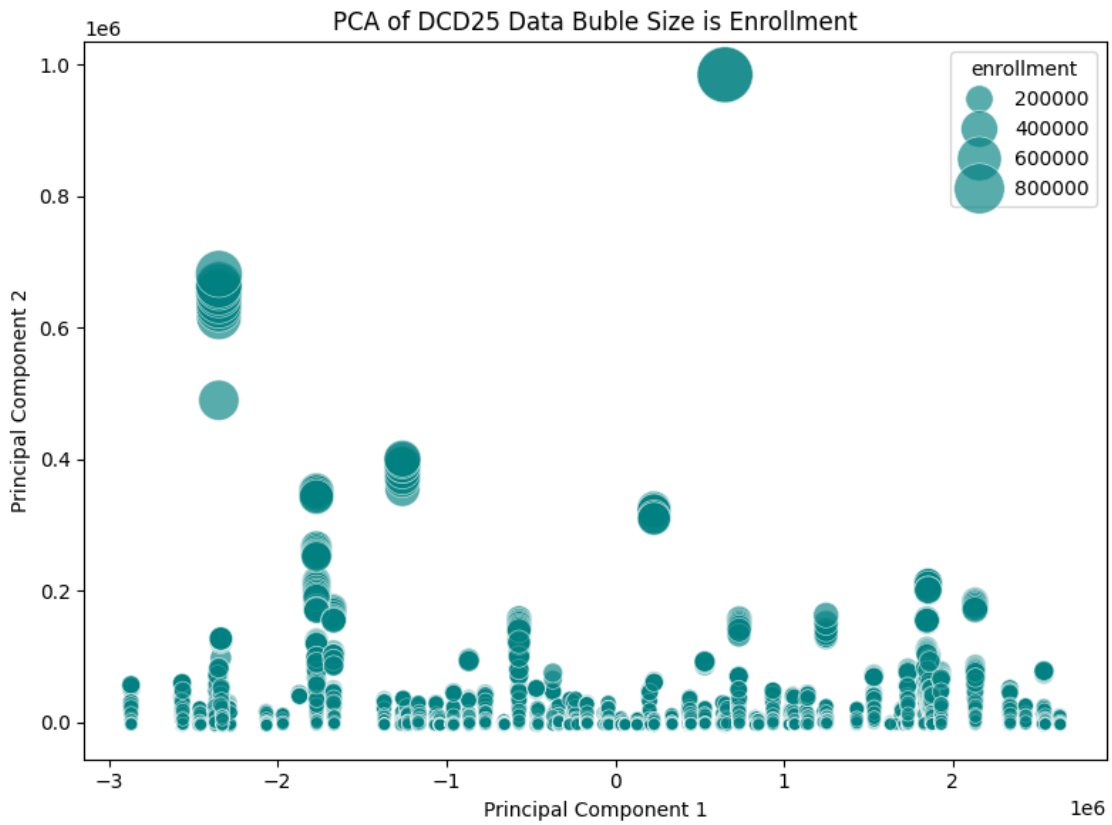

Bubble size = Difference between Required and Actual Spending
Required Actual Difference
Test Score Gap
0
20000
40000
60000
80000
100000
0
20000
40000
60000
80000

[13]:
```
fig, ax = plt.subplots(figsize=(6, 10))

sns.scatterplot(
    data=dcd25,
    x="per_hispanic",
    y="per_eng_learn",
    size="req_actual_diff",
    sizes=(40, 800),
    alpha=0.65,
    color="teal",
    legend=False,
    ax=ax
)

ax.set_title("Bubble size = Difference between Required and Actual Spending")
ax.set_xlabel("Percent Hispanic")
ax.set_ylabel("Percent English Learners")

plt.tight_layout()
plt.show()
```


Bubble size = Difference between Required and Actual Spending
Percent English Learners
Percent Hispanic
0.0
0.2
0.4
0.6
0.8
1.0

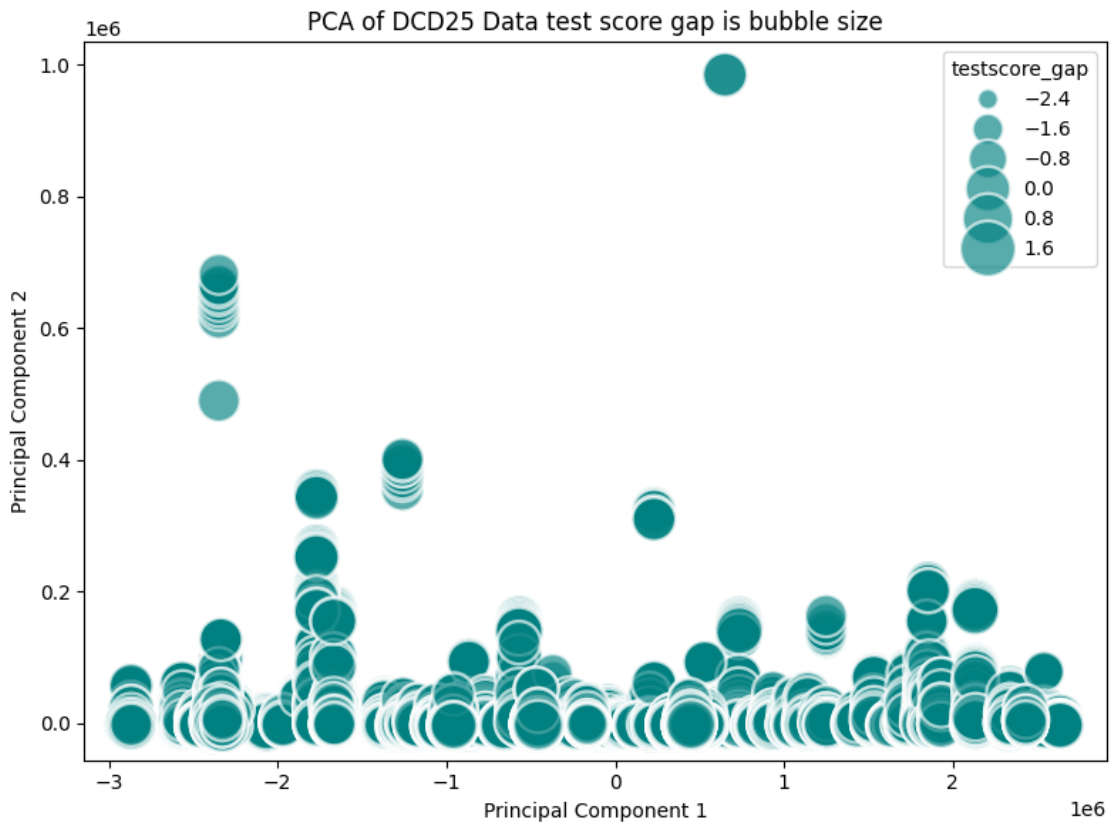
PCA plots with legends

[14]:
```
dcd25_dropped = dcd25.dropna()  # drop NaN values for PCA
pca = decomposition.PCA(n_components=2)
pca.fit(dcd25_dropped.select_dtypes(include=[np.number]))  # fit PCA on numeric␣
↪columns
pca_scores = pca.transform(dcd25_dropped.select_dtypes(include=[np.number]))  #␣
↪get PCA scores
pca_df = pd.DataFrame(pca_scores, columns=['PC1', 'PC2'])  # create DataFrame␣
↪for PCA scores
pca_df['enrollment'] = dcd25_dropped['enrollment'].values
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    size='enrollment',
    sizes=(40, 800),
    alpha=0.65,
    color='teal',
    legend=True
)
plt.title("PCA of DCD25 Data Buble Size is Enrollment")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.tight_layout()
plt.show()
```



[15]:
```
# do PCA on dcd25, then plot the first two principal components with bubble␣
↪size = testscore_gap
dcd25_dropped = dcd25.dropna()  # drop NaN values for PCA
pca = decomposition.PCA(n_components=2)
pca.fit(dcd25_dropped.select_dtypes(include=[np.number]))  # fit PCA on numeric␣
↪columns
pca_scores = pca.transform(dcd25_dropped.select_dtypes(include=[np.number]))  #␣
↪get PCA scores
pca_df = pd.DataFrame(pca_scores, columns=['PC1', 'PC2'])  # create DataFrame␣
↪for PCA scores
pca_df['testscore_gap'] = dcd25_dropped['testscore_gap'].values
plt.figure(figsize=(8, 6))
sns.scatterplot(
    data=pca_df,
    x='PC1',
    y='PC2',
    size='testscore_gap',
    sizes=(40, 800),
    alpha=0.65,
    color='teal',
```

```
    legend=True
)
plt.title("PCA of DCD25 Data test score gap is bubble size")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.tight_layout()
plt.show()
```

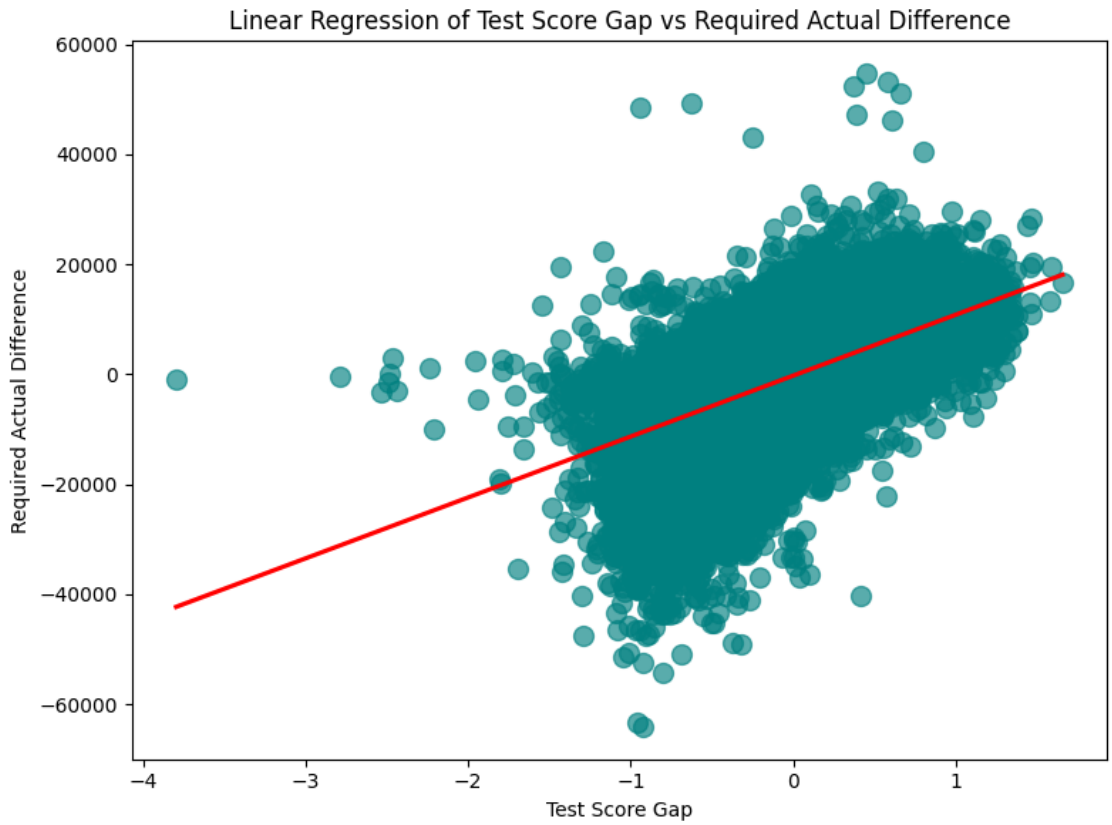


Linear Regression plots

*** Linear regression analysis of testscore_gap vs req_actual_diff**

[16]:
```
# linear regression analysis of testscore_gap vs req_actual_diff
plt.figure(figsize=(8, 6))
sns.regplot(
    data=dcd25,
    x="testscore_gap",
    y="req_actual_diff",
    scatter_kws={"s": 100, "alpha": 0.65, "color": "teal"},
    line_kws={"color": "red"},
)
```

```
plt.title("Linear Regression of Test Score Gap vs Required Actual Difference")
plt.xlabel("Test Score Gap")
plt.ylabel("Required Actual Difference")
plt.tight_layout()
plt.show()
```

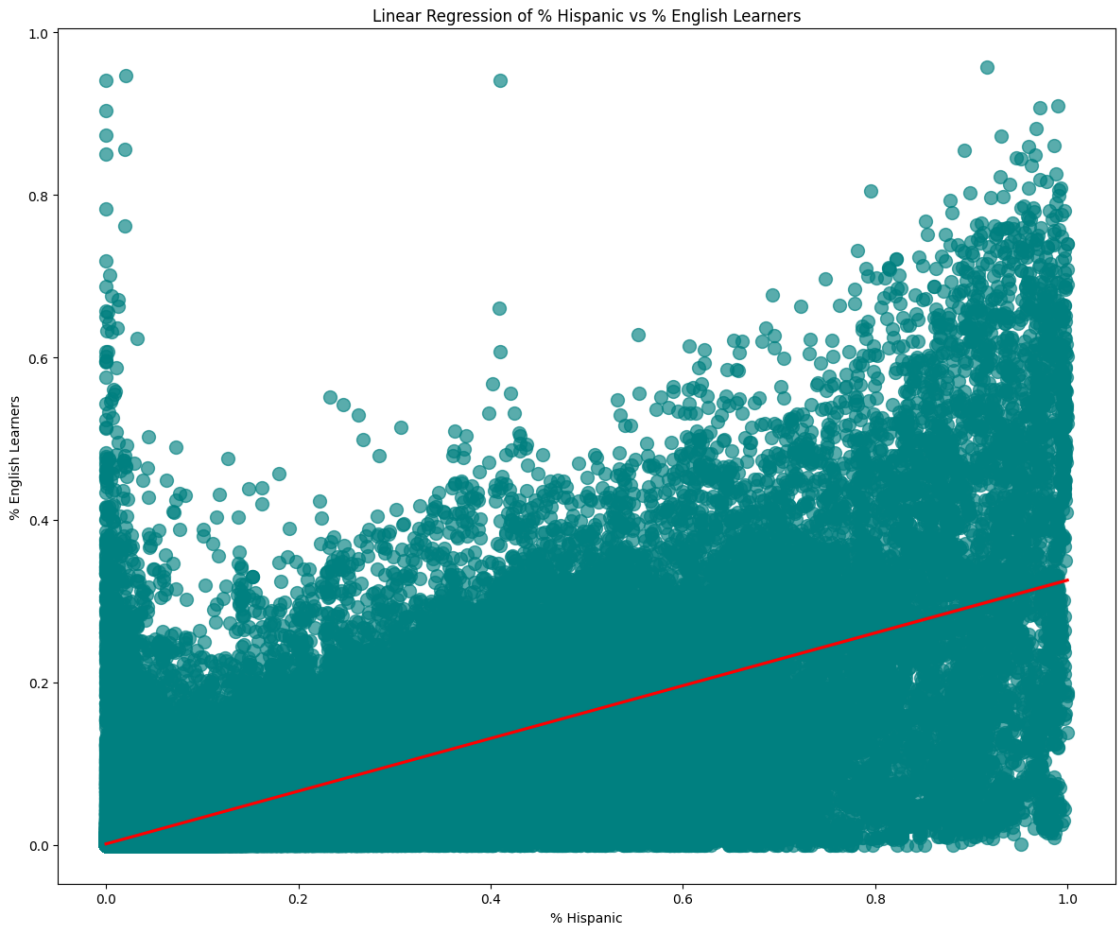


There definitely seems to be a linear correlation between these two variables.

*** Linear regression of percent hispanic and percent english language learners**

[17]:
```
# linear regression analysis per_hispanic and per_english_learners
plt.figure(figsize=(12, 10))
sns.regplot(
    data=dcd25_renamed,
    x="per_hispanic",
    y="per_eng_learn",
    scatter_kws={"s": 100, "alpha": 0.65, "color": "teal"},
    line_kws={"color": "red"},
)
plt.title("Linear Regression of % Hispanic vs % English Learners")
plt.xlabel("% Hispanic")
plt.ylabel("% English Learners")
```

```
plt.tight_layout()
plt.show()
```

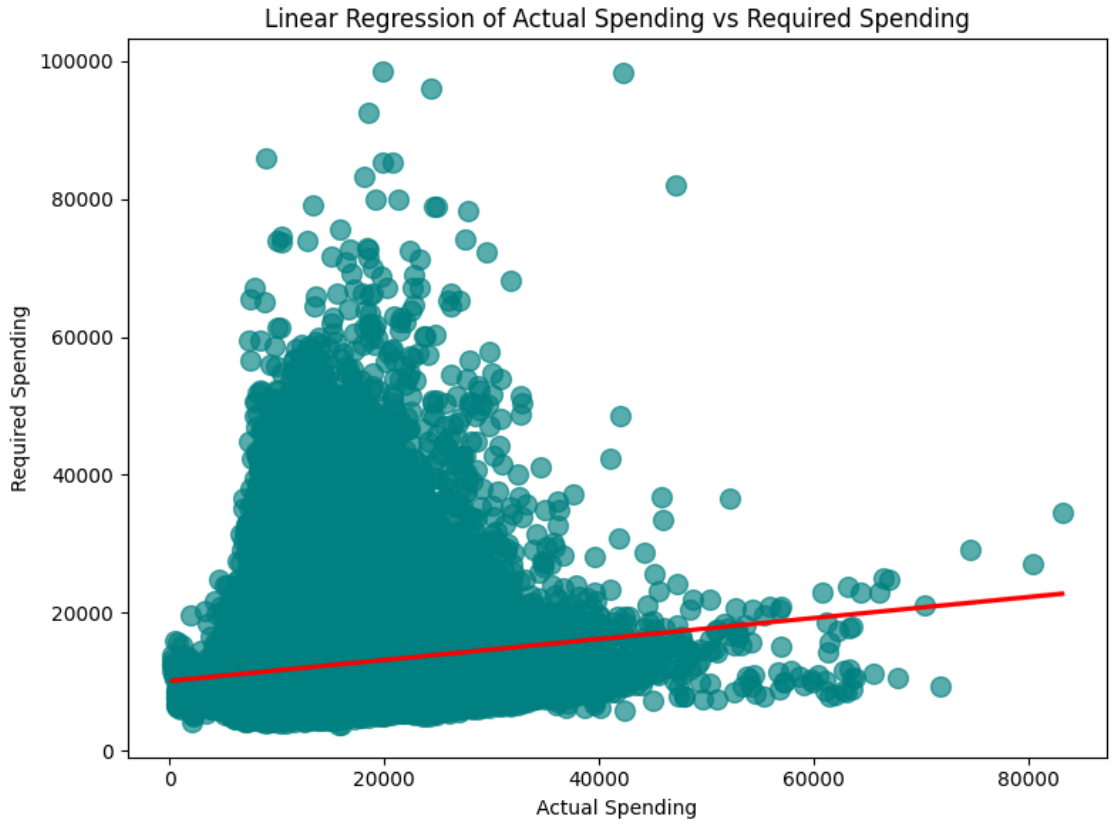


This pair seems like it would be highly correlated given the numbers in the correlation matrix. The linear regression plot though seems to indicate a basic correlation, but not one as intense as the one indicated in the correlation matrix.

Linear regression of Actual Spending versus Required spending

[18]:
```
# linear regression analysis of testscore_gap vs req_actual_diff
plt.figure(figsize=(8, 6))
sns.regplot(
    data=dcd25,
    x="actual_spending",
    y="req_spending",
    scatter_kws={"s": 100, "alpha": 0.65, "color": "teal"},
    line_kws={"color": "red"},
)
plt.title("Linear Regression of Actual Spending vs Required Spending")
```

```
plt.xlabel("Actual Spending")
plt.ylabel("Required Spending")
plt.tight_layout()
plt.show()
```

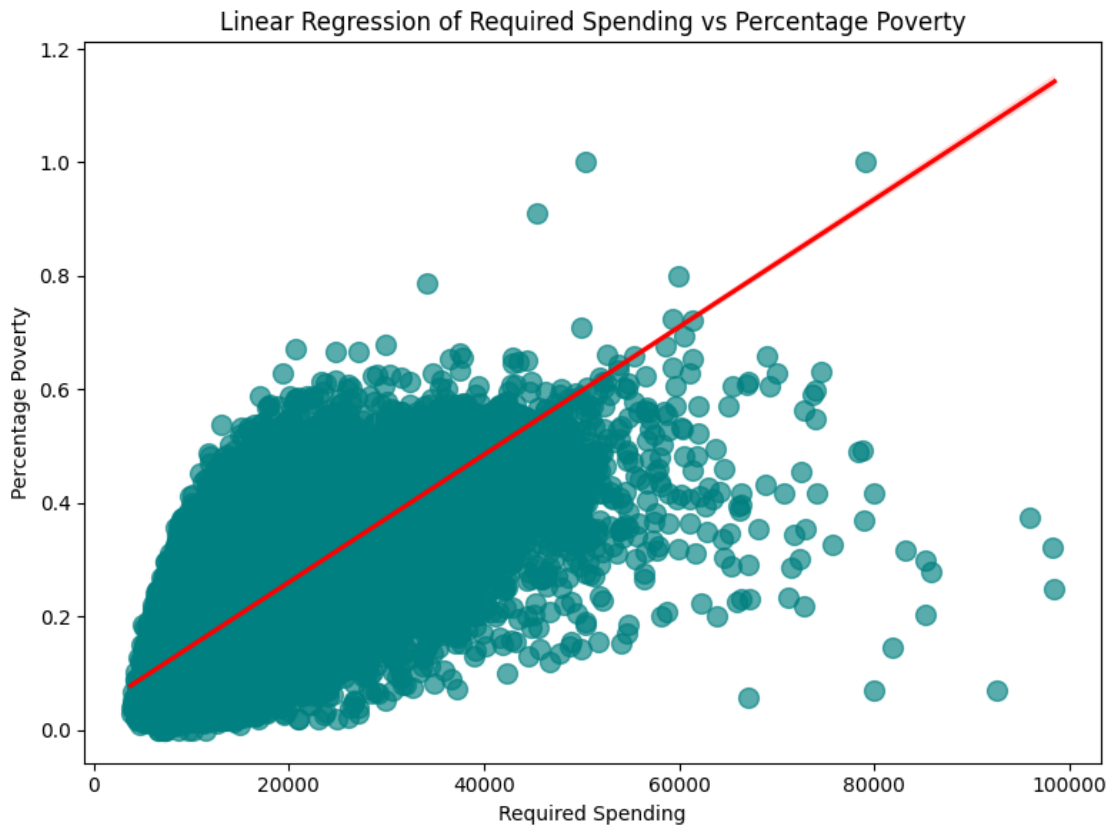


Linear regression of testscore gap versus actual spending

[19]:
```
# linear regression analysis of testscore_gap vs req_actual_diff
plt.figure(figsize=(8, 6))
sns.regplot(
    data=dcd25,
    x="testscore_gap",
    y="actual_spending",
    scatter_kws={"s": 100, "alpha": 0.65, "color": "teal"},
    line_kws={"color": "red"},
)
plt.title("Linear Regression of Test Score Gap vs Actual Spending")
plt.xlabel("Test Score Gap")
plt.ylabel("Actual Spending")
plt.tight_layout()
plt.show()
```

*** linear regression of required soending versus percentage of students in poverty**

[20]:
```
# linear regression
plt.figure(figsize=(8, 6))
sns.regplot(
    data=dcd25,
    x="req_spending",
    y="per_poverty",
    scatter_kws={"s": 100, "alpha": 0.65, "color": "teal"},
    line_kws={"color": "red"},
)
plt.title("Linear Regression of Required Spending vs Percentage Poverty")
plt.xlabel("Required Spending")
plt.ylabel("Percentage Poverty")
plt.tight_layout()
plt.show()
```

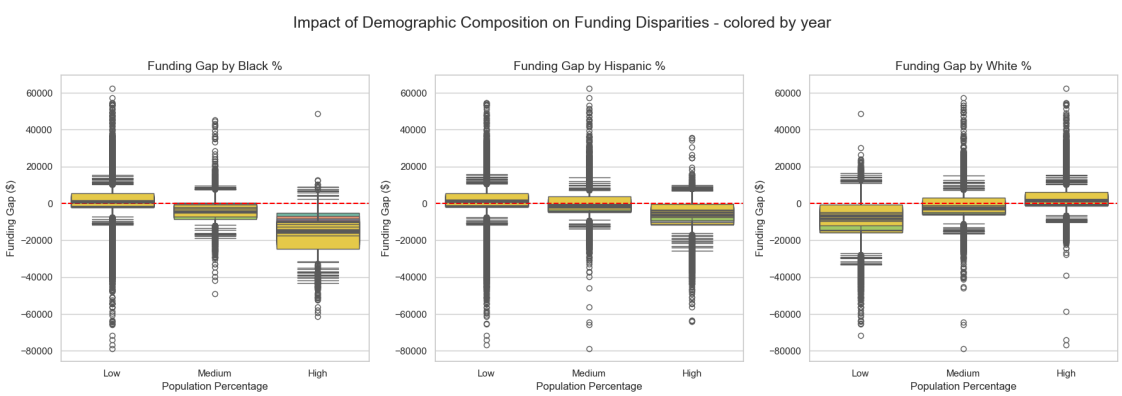


This pair not only seems like they should be correlated (the percent of students in poverty is likely to be correlated to the amount of money a district is required to spend per student), it also seems like the linear regression indicates that correlation.

4.0.1 further analysis with other features

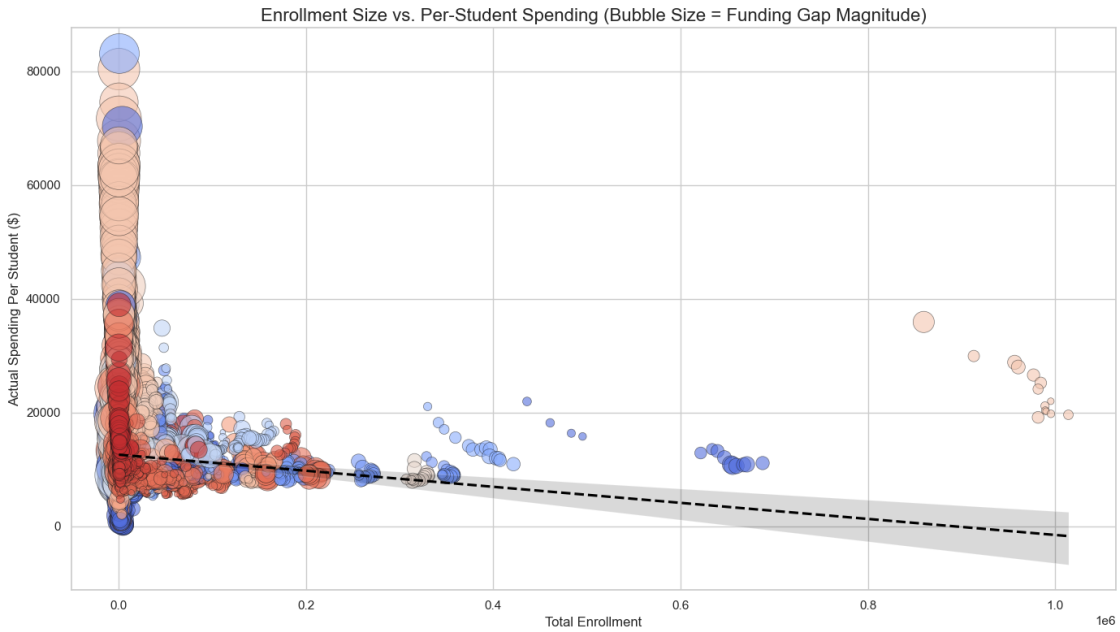
[22]:
```
# Clean data: Convert empty strings to NaN and drop rows with missing critical␣
↪values
df = dcd25_renamed.copy()  # work with a copy of the original DataFrame
df.replace('', pd.NA, inplace=True)
df['req_actual_diff'] = pd.to_numeric(df['req_actual_diff'], errors='coerce')
df['per_poverty'] = pd.to_numeric(df['per_poverty'], errors='coerce')
df['enrollment'] = pd.to_numeric(df['enrollment'], errors='coerce')
df['actual_spending'] = pd.to_numeric(df['actual_spending'], errors='coerce')

# Drop rows where critical analysis columns are missing
df_clean = df.dropna(subset=['req_actual_diff', 'per_poverty', 'enrollment'])

# Set style
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (12, 8)
```

[24]:
```
# Demographic Composition vs. Funding Gap (Boxplots)
# We'll focus on Black, Hispanic, and White populations as primary drivers
demographic_cols = ['per_black', 'per_hispanic', 'per_white']
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

for i, col in enumerate(demographic_cols):
    # Filter out rows with missing demographic data
    subset = df_clean.dropna(subset=[col])

    # Create a categorical bin for the demographic percentage to make boxplots␣
↪readable
    # e.g., Low (<20%), Medium (20-60%), High (>60%)
    bins = [0, 0.2, 0.6, 1.0]
    labels = ['Low', 'Medium', 'High']
    subset['demographic_bin'] = pd.cut(subset[col], bins=bins, labels=labels,␣
↪include_lowest=True)

    sns.boxplot(x='demographic_bin', y='req_actual_diff', data=subset,␣
↪ax=axes[i], palette='Set2', hue='year', dodge=False, legend=False)
    axes[i].set_title(f'Funding Gap by {col.replace("per_", "").replace("_", "␣
↪").title()} %', fontsize=14)
    axes[i].axhline(0, color='red', linestyle='--', linewidth=1.5)
    axes[i].set_xlabel('Population Percentage')
    axes[i].set_ylabel('Funding Gap ($)')

plt.suptitle('Impact of Demographic Composition on Funding Disparities -␣
↪colored by year', fontsize=18, y=1.02)
plt.tight_layout()
plt.show()
```



It is significant that the funding gap gets much worse when the students' skin color is not "white." Students that are designated in the "black" group seem to have a significantly greater gap in funding.

[25]:
```
# Enrollment vs. Spending Efficiency (Bubble Chart)
plt.figure(figsize=(14, 8))
# Calculate absolute gap for sizing (magnitude of the problem regardless of␣
↪direction)
df_clean['gap_magnitude'] = df_clean['req_actual_diff'].abs()

bubble = sns.scatterplot(
    data=df_clean,
    x='enrollment',
    y='actual_spending',
    size='gap_magnitude',
    hue='state_name',
    sizes=(20, 2000), # Scale bubble sizes
    palette='coolwarm',
    alpha=0.6,
    legend=False,
    edgecolor='k'
)

# Add a trend line for spending vs enrollment
sns.regplot(data=df_clean, x='enrollment', y='actual_spending', scatter=False,␣
↪color='black', line_kws={'linestyle':'--'})

plt.title('Enrollment Size vs. Per-Student Spending (Bubble Size = Funding Gap␣
↪Magnitude)', fontsize=16)
plt.xlabel('Total Enrollment', fontsize=12)
plt.ylabel('Actual Spending Per Student ($)', fontsize=12)
plt.tight_layout()
plt.show()
```

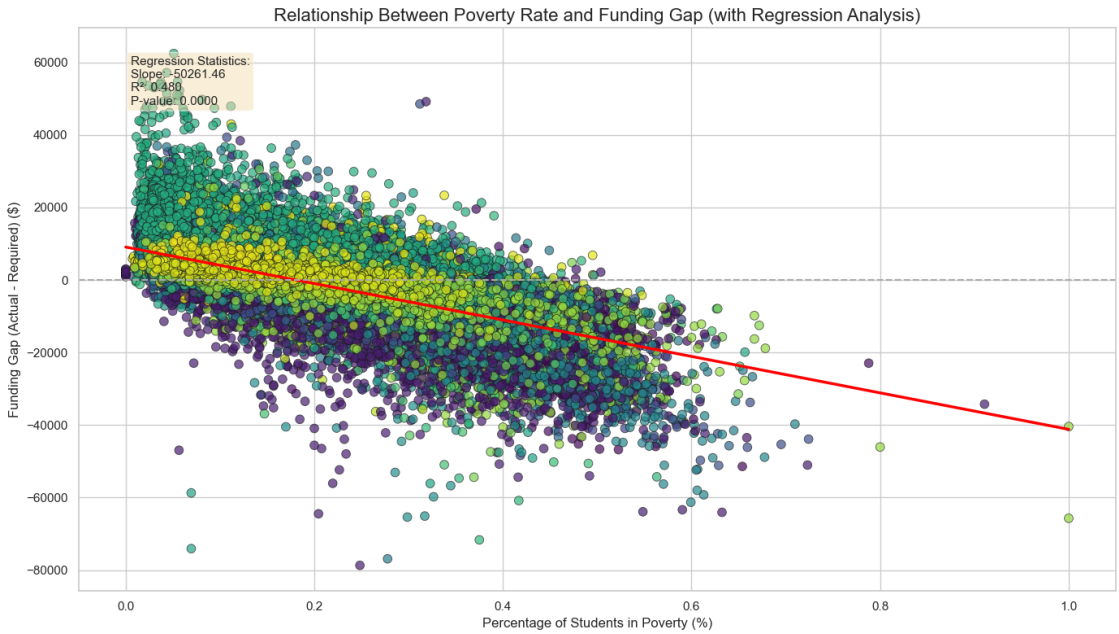


[32]:
```
from scipy import stats
```

[33]:
```
# Drop rows where critical analysis columns are missing
df_clean = df.dropna(subset=['req_actual_diff', 'per_poverty', 'enrollment'])

# Set style
sns.set(style="whitegrid")
plt.rcParams['figure.figsize'] = (14, 8)

# Spending Gap vs. Poverty Rate WITH Regression Line
plt.figure(figsize=(14, 8))

# Create scatter plot
scatter = sns.scatterplot(
    data=df_clean,
    x='per_poverty',
    y='req_actual_diff',
    hue='state_name',
    palette='viridis',
    alpha=0.7,
    s=60,
    edgecolor='k',
    legend=False
)

# Add regression line (using all data points, not colored by state)
```

```python
reg_line = sns.regplot(
    data=df_clean,
    x='per_poverty',
    y='req_actual_diff',
    scatter=False,  # Don't show the scatter points again
    color='red',
    line_kws={'linewidth': 2.5, 'label': 'Regression Line'},
    ci=95,  # Show 95% confidence interval
)

# Calculate and display R-squared value
slope, intercept, r_value, p_value, std_err = stats.linregress(
    df_clean['per_poverty'],
    df_clean['req_actual_diff']
)
r_squared = r_value ** 2

# Add text annotation with regression statistics
text_str = f'Regression Statistics:\nSlope: {slope:.2f}\nR²: {r_squared:.
 ↪3f}\nP-value: {p_value:.4f}'
plt.text(0.05, 0.95, text_str, transform=plt.gca().transAxes,
         fontsize=11, verticalalignment='top',
         bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.title('Relationship Between Poverty Rate and Funding Gap (with Regression␣
 ↪Analysis)', fontsize=16)
plt.xlabel('Percentage of Students in Poverty (%)', fontsize=12)
plt.ylabel('Funding Gap (Actual - Required) ($)', fontsize=12)
plt.axhline(0, color='darkgray', linestyle='--', linewidth=1.5,␣
 ↪label='Break-even Point')
# plt.legend(title='State')
plt.tight_layout()
plt.show()

# Optional: Print detailed regression output
print("=" * 60)
print("REGRESSION ANALYSIS SUMMARY")
print("=" * 60)
print(f"Dependent Variable: Funding Gap (req_actual_diff)")
print(f"Independent Variable: Poverty Rate (per_poverty)")
print(f"\nRegression Equation: y = {intercept:.2f} + ({slope:.2f})x")
print(f"R-squared: {r_squared:.3f}")
print(f"P-value: {p_value:.4f}")
print(f"\nInterpretation:")
print(f"- For every 1% increase in poverty rate, funding gap changes by ${slope:
 ↪.2f}")
```

```
print(f"- R² of {r_squared:.3f} indicates {'strong' if r_squared > 0.5 else␣
↪'moderate' if r_squared > 0.2 else 'weak'} correlation")
print(f"- P-value {'< 0.05 (statistically significant)' if p_value < 0.05 else␣
↪'> 0.05 (not statistically significant)'}")
print("=" * 60)
```



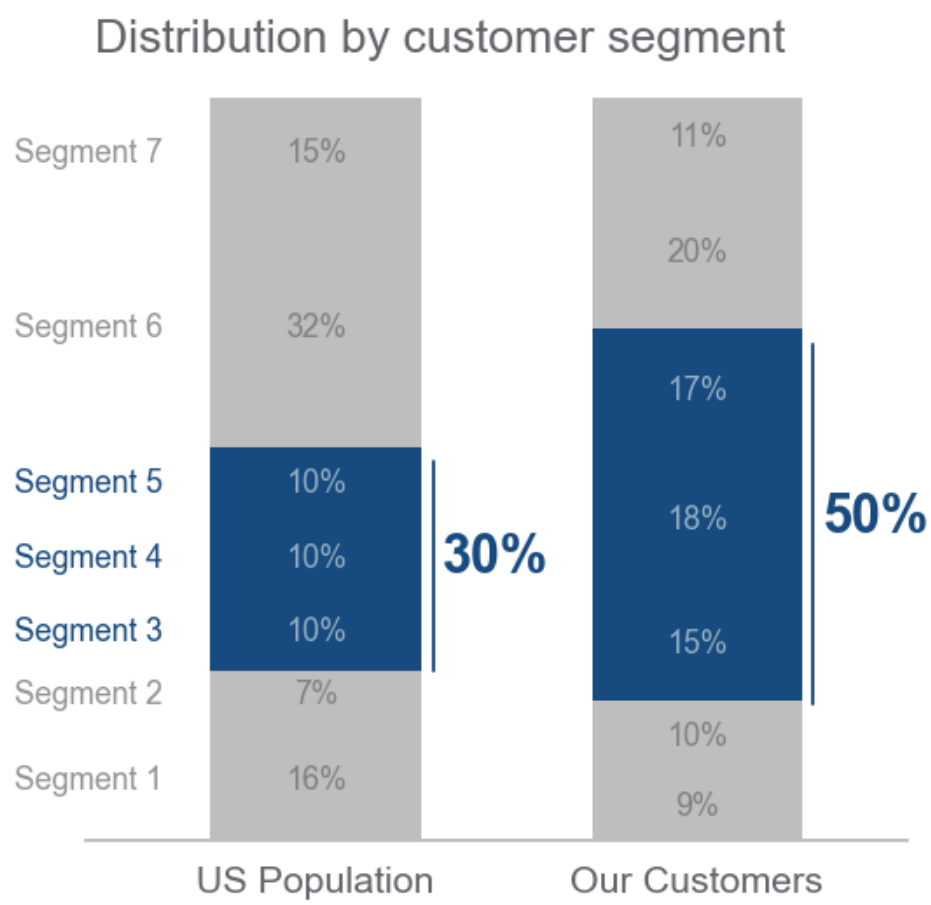

```
============================================================
REGRESSION ANALYSIS SUMMARY
============================================================
Dependent Variable: Funding Gap (req_actual_diff)
Independent Variable: Poverty Rate (per_poverty)

Regression Equation: y = 9088.45 + (-50261.46)x
R-squared: 0.480
P-value: 0.0000

Interpretation:
- For every 1% increase in poverty rate, funding gap changes by $-50261.46
- R² of 0.480 indicates moderate correlation
- P-value < 0.05 (statistically significant)
============================================================
```

This chart seems to show that the greater the percentage of students in poverty, the larger the gap in funding. Specifically, as more students are in the poverty category, there is a significant drop in funding, when the opposite should be true.

5 4. Storytelling With Data plot

Reproduce any graph of your choice in p. 136-150 of the Storytelling With Data book as best you can. ("The power of super-categories" to the end of chapter 5). You do not have to get the exact data values right, just the overall look and feel.

Reproduction of figure 5.13 on page 148 of Storytelling with Data

[ ]:
```
import numpy as np
import seaborn as sns

# Data
segments = [f"Segment {i}" for i in range(1, 8)]
us = np.array([16, 7, 10, 10, 10, 32, 15])
cust = np.array([9, 10, 15, 18, 17, 20, 11])

# Colors
GRAY4, GRAY6, GRAY7, GRAY9 = "#646369", "#828282", "#929497", "#BFBEBE"
BLUE1, BLUE4 = "#174A7E", "#94AFC5"
block_colors = [GRAY9]*2 + [BLUE1]*3 + [GRAY9]*2
value_colors = [GRAY6]*2 + [BLUE4]*3 + [GRAY6]*2
label_colors = [GRAY7]*2 + [BLUE1]*3 + [GRAY7]*2

# Theme
sns.set_theme(style="white", font="Arial")

fig, ax = plt.subplots(figsize=(7, 5), dpi=110)
fig.subplots_adjust(left=0.16, right=0.77, top=0.89, bottom=0.1)

bottom = np.zeros(2)
for i, seg in enumerate(segments):
    vals = np.array([us[i], cust[i]])
    bars = ax.bar([0, 1], vals, bottom=bottom, color=block_colors[i], width=0.
 ↪55, linewidth=0)

    ax.text(-0.4, bottom[0] + us[i] / 2, seg, ha="right", va="center",␣
 ↪fontsize=11.5, color=label_colors[i])

    for j, b in enumerate(bars):
        ax.text(
            b.get_x() + b.get_width()/2,
            b.get_y() + b.get_height()/2,
            f"{vals[j]}%",
            ha="center", va="center", fontsize=11.5, color=value_colors[i]
        )
    bottom += vals

ax.set_xlim(-0.6, 1.55)
```

```
ax.set_ylim(0, 102)
ax.set_xticks([0, 1], ["US Population", "Our Customers"])
ax.tick_params(bottom=False, left=False, labelleft=False)
for s in ["left", "right", "top"]:
    ax.spines[s].set_visible(False)
ax.spines["bottom"].set_color(GRAY9)
for t in ax.get_xticklabels():
    t.set_fontsize(14)
    t.set_color(GRAY4)

ax.text(-0.58, 106, "Distribution by customer segment", color=GRAY4,␣
 ↪fontsize=17.4)
ax.axvline(0.31, ymin=0.225, ymax=0.5, color=BLUE1, linewidth=1.2)
ax.axvline(1.3, ymin=0.18, ymax=0.655, color=BLUE1, linewidth=1.2)
ax.text(0.33, 36, "30%", fontweight="bold", fontsize=20, color=BLUE1)
ax.text(1.33, 41.5, "50%", fontweight="bold", fontsize=20, color=BLUE1)

plt.savefig("SWD_p148_segment_distribution.png", dpi=300, bbox_inches="tight")
plt.show()
```